

Proyecto Cuatrimestral con IA - Corte I

TraceIT - Mesa de ayuda informática

Planteamiento, base funcional y uso responsable de la IA

Integrante	Johan Rosas
Tecnología	Python + Flask + SQLAlchemy + SQLite + HTML/CSS
Repositorio	https://github.com/johanr-lab/TraceIT-CorteI (publicar antes de la entrega)
Prototipo	Archivo aparte: Prototipo_CorteI_TraceIT.pdf
Fecha de elaboración	4 de octubre de 2026

1. Problema

En una mesa de ayuda informática universitaria, las solicitudes de soporte pueden llegar por diferentes medios como mensajes, correo o conversaciones directas. Cuando no existe un registro centralizado, es fácil perder el seguimiento de un incidente, repetir diagnósticos y no saber con claridad qué casos están abiertos, en proceso o resueltos. Esto afecta tanto a estudiantes y docentes que esperan una respuesta como al personal de tecnología que necesita organizar su trabajo y conservar el historial de atención.

TraceIT busca resolver este problema mediante una aplicación web sencilla para registrar y administrar tickets de soporte. Cada ticket queda asociado a un usuario y a una categoría, además de almacenar una descripción, prioridad y estado. En el Corte I el objetivo no es automatizar la decisión técnica, sino construir la base funcional del sistema y garantizar que la información quede organizada y persistente.

El proyecto está dirigido principalmente a mesas de ayuda pequeñas o equipos de TI universitarios que necesitan una herramienta fácil de usar, con una estructura suficientemente clara para crecer durante los siguientes cortes. Más adelante, la Inteligencia Artificial podrá apoyar tareas como clasificación, resumen o búsqueda de incidentes similares, siempre como asistencia al técnico y no como reemplazo de su criterio.

2. Requisitos funcionales

ID	Requisito funcional
RF-01	Registrar un ticket con asunto, descripción, usuario, categoría, prioridad y estado.
RF-02	Listar los tickets almacenados mostrando la información principal de cada caso.
RF-03	Consultar el detalle completo de un ticket seleccionado.
RF-04	Editar la información de un ticket existente y guardar los cambios.
RF-05	Eliminar un ticket cuando ya no deba permanecer en el sistema.
RF-06	Relacionar cada ticket con un usuario registrado.
RF-07	Relacionar cada ticket con una categoría de soporte.
RF-08	Manejar prioridades Baja, Media y Alta, y estados Abierto, En proceso y Resuelto.
RF-09	Filtrar el listado de tickets por estado.
RF-10	Persistir los datos en una base SQLite para conservarlos entre ejecuciones.

3. Requisitos no funcionales

ID	Tipo	Descripción
RNF-01	Usabilidad	La interfaz debe ser comprensible para un usuario con conocimientos básicos de navegación web.
RNF-02	Rendimiento	Las operaciones del CRUD deben responder de forma fluida en un entorno local de demostración.
RNF-03	Integridad	Los campos obligatorios y los valores permitidos deben validarse antes de guardar información.
RNF-04	Mantenibilidad	El código debe estar separado en modelos, rutas, plantillas y archivos estáticos para facilitar cambios.
RNF-05	Portabilidad	El proyecto debe poder ejecutarse en macOS, Windows o Linux con Python y sus dependencias instaladas.
RNF-06	Privacidad	El Corte I no almacenará contraseñas ni datos sensibles; solo información básica necesaria para el ticket.

4. Modelo de datos

El modelo utiliza tres entidades principales. Ticket es la entidad central y contiene las llaves foráneas que lo relacionan con Usuario y Categoría. Esto evita interpretar el modelo como una cadena Usuario → Ticket → Categoría: en realidad son dos relaciones independientes de uno a muchos.

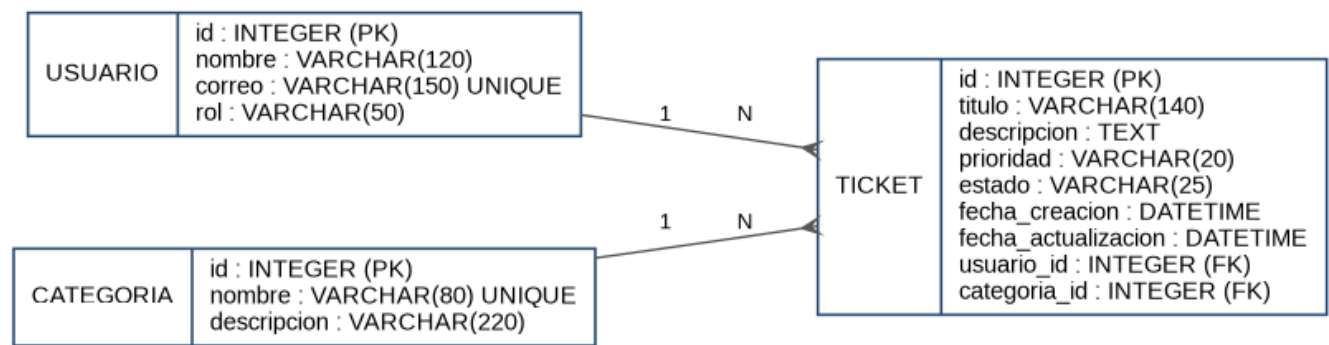


Figura 1. Modelo entidad-relación de TraceIT.

Entidad	Propósito	Datos principales
Usuario	Representa a quien registra o está asociado a un incidente.	id, nombre, correo, rol
Categoría	Permite clasificar los tickets por tipo de soporte.	id, nombre, descripción
Ticket	Almacena y da seguimiento a la solicitud de soporte.	id, título, descripción, prioridad, estado, fechas, usuario_id, categoria_id

5. Estado del módulo funcional

- CRUD completo de Ticket: crear, consultar, editar y eliminar.
- Base SQLite configurada para generarse automáticamente al iniciar la aplicación.
- Usuarios y categorías de referencia para poder probar el CRUD desde el primer inicio.
- Validaciones de campos obligatorios, prioridades, estados y relaciones.
- Pruebas automatizadas incluidas para el flujo principal del CRUD.

Nota para la entrega: el repositorio local ya contiene historial de commits. Antes de entregar al docente debe publicarse en GitHub bajo la cuenta johanr-lab y comprobar que el enlace sea accesible.